

Salesforce Apex Test Classes



explain test class in salesforce apex in detail with multiple examples and explain all the examples

In **Salesforce Apex**, a **test class** is used to verify that your code behaves as expected. Salesforce requires at least **75% code coverage** through test methods before deploying Apex code to a production environment.

◆ Why Use Test Classes in Apex?

- Ensure code correctness.
- Validate logic for different scenarios.
- Avoid breaking changes during future deployments.
- Required for deployment to production.

◆ Key Annotations in Test Classes

Annotation	Purpose
<code>@isTest</code>	Marks a class or method as a test
<code>@testSetup</code>	Creates common test data for all test methods
<code>Test.startTest()</code> / <code>Test.stopTest()</code>	Helps test governor limits by resetting them

◆ Syntax of a Basic Test Class

```
apex

@isTest
private class MyTestClass {
    @isTest
    static void myTestMethod() {
        // Your test logic here
    }
}
```

◆ Example 1: Test Class for a Simple Trigger

Trigger

```
apex

trigger AccountTrigger on Account (before insert) {
    for(Account acc : Trigger.new){
        acc.Name = acc.Name + ' - Validated';
    }
}
```

Test Class

```
apex

@isTest
private class TestAccountTrigger {
    @isTest
    static void testAccountNameModification() {
        Account acc = new Account(Name = 'TestAccount');
        insert acc;

        Account insertedAcc = [SELECT Name FROM Account WHERE Id = :acc.Id];
        System.assertEquals('TestAccount - Validated', insertedAcc.Name);
    }
}
```

Explanation:

- Inserts an Account.
- The trigger modifies its name.
- We fetch and verify that the name was changed as expected.

◆ Example 2: Test Class with @testSetup and DML operations

Apex Class

```
apex

public class OpportunityHandler {
    public static void updateStage(List<Opportunity> opps) {
        for(Opportunity opp : opps){
            opp.StageName = 'Closed Won';
        }
        update opps;
    }
}
```

Test Class

```
apex

@isTest
private class TestOpportunityHandler {

    @testSetup
    static void setupData() {
        Account acc = new Account(Name = 'TestAcc');
        insert acc;
    }
}
```

```

        Opportunity opp = new Opportunity(
            Name = 'TestOpp',
            CloseDate = Date.today(),
            StageName = 'Prospecting',
            AccountId = acc.Id
        );
        insert opp;
    }

    @isTest
    static void testUpdateStage() {
        List<Opportunity> opps = [SELECT Id, StageName FROM Opportunity];

        Test.startTest();
        OpportunityHandler.updateStage(opps);
        Test.stopTest();

        for (Opportunity opp : [SELECT StageName FROM Opportunity]) {
            System.assertEquals('Closed Won', opp.StageName);
        }
    }
}

```

Explanation:

- **@testSetup** creates reusable test data.
- **updateStage** is called within **startTest()** and **stopTest()** block to simulate real behavior and test limits.
- We assert that the **StageName** changed.

Example 3: Test Class for a Class with SOQL Query

Apex Class

```

apex

public class ContactFetcher {
    public static List<Contact> getContactsForAccount(Id accountId) {
        return [SELECT Id, LastName FROM Contact WHERE AccountId = :accountId];
    }
}

```

Test Class

```

apex

@isTest
private class TestContactFetcher {
    @isTest
    static void testGetContactsForAccount() {
        Account acc = new Account(Name = 'TestAccount');
        insert acc;

        Contact con = new Contact(LastName = 'Test', AccountId = acc.Id);
        insert con;

        List<Contact> contacts = ContactFetcher.getContactsForAccount(acc.Id);
        System.assertEquals(1, contacts.size());
        System.assertEquals('Test', contacts[0].LastName);
    }
}

```

```
}
}
```

Explanation:

- Inserts an Account and related Contact.
- Calls the method and checks that it retrieves the correct Contact.

◆ Example 4: Test Class for Exception Handling

Apex Class

```
apex

public class DivisionHandler {
    public static Integer divide(Integer a, Integer b) {
        if (b == 0) {
            throw new ArithmeticException('Cannot divide by zero');
        }
        return a / b;
    }
}
```

Test Class

```
apex

@Test
private class TestDivisionHandler {
    @Test
    static void testDivideValid() {
        Integer result = DivisionHandler.divide(10, 2);
        System.assertEquals(5, result);
    }

    @Test
    static void testDivideByZero() {
        try {
            DivisionHandler.divide(10, 0);
            System.assert(false, 'Expected an exception');
        } catch (Exception e) {
            System.assert(e instanceof ArithmeticException);
            System.assertEquals('Cannot divide by zero', e.getMessage());
        }
    }
}
```

Explanation:

- Tests both valid and invalid division.
- Ensures the code gracefully handles exceptions.

◆ Example 5: Test Class for Apex Batch Class

Batch Class

apex

```
global class AccountBatch implements Database.Batchable<SObject> {
    global Database.QueryLocator start(Database.BatchableContext BC) {
        return Database.getQueryLocator('SELECT Id FROM Account');
    }

    global void execute(Database.BatchableContext BC, List<SObject> scope) {
        for (SObject s : scope) {
            Account acc = (Account)s;
            acc.Name += ' - Processed';
        }
        update scope;
    }

    global void finish(Database.BatchableContext BC) {}
}
```

Test Class

apex

```
@isTest
private class TestAccountBatch {
    @isTest
    static void testBatchExecution() {
        List<Account> accs = new List<Account>();
        for (Integer i = 0; i < 5; i++) {
            accs.add(new Account(Name = 'Acc ' + i));
        }
        insert accs;

        Test.startTest();
        AccountBatch batch = new AccountBatch();
        Database.executeBatch(batch, 2);
        Test.stopTest();

        for (Account acc : [SELECT Name FROM Account]) {
            System.assert(acc.Name.contains('Processed'));
        }
    }
}
```

Explanation:

- Inserts 5 test Accounts.
- Executes the batch and checks that names have been updated.

Best Practices

- Use `@testSetup` to avoid repeated code.
- Use `System.assert()` to validate behavior.
- Cover both positive and negative scenarios.
- Always cleanly separate test methods for clarity.

If you want me to provide examples for **Triggers**, **Future methods**, **Queueables**, **Scheduled Apex**, or **REST API**, I can share those as well.

Would you like more examples on those?



yes

Great! Let's now cover **test classes for advanced Apex scenarios**, including:

1. ☒ Triggers with logic classes
2. ☒ Future methods
3. ☒ Queueable Apex
4. ☒ Scheduled Apex
5. ☒ REST API callouts with `HttpCalloutMock`

◆ 1. Trigger with Handler Class

◆ Trigger

```
apex

trigger ContactTrigger on Contact (before insert) {
    ContactHandler.validateContacts(Trieger.new);
}
```

◆ Handler Class

```
apex

public class ContactHandler {
    public static void validateContacts(List<Contact> contacts) {
        for (Contact c : contacts) {
            if (String.isBlank(c.LastName)) {
                c.LastName = 'DefaultLastName';
            }
        }
    }
}
```

◆ Test Class

```
apex

@isTest
private class TestContactTrigger {
    @isTest
    static void testContactHandlerLogic() {
        Contact c = new Contact(FirstName = 'Test');
        insert c;

        Contact inserted = [SELECT LastName FROM Contact WHERE Id = :c.Id];
        System.assertEquals('DefaultLastName', inserted.LastName);
    }
}
```

```
}
}
```

✓ Tests that the trigger sets a default last name if missing.

◆ 2. Future Method Test

◆ Future Class

```
apex

public class AsyncLogger {
    @future
    public static void logAsync(String message) {
        System.debug('Log: ' + message);
    }
}
```

◆ Test Class

```
apex

@isTest
private class TestAsyncLogger {
    @isTest
    static void testLogAsync() {
        Test.startTest();
        AsyncLogger.logAsync('Hello future');
        Test.stopTest();

        // No assert needed as it's just debug, but test ensures method runs
    }
}
```

✓ Tests a future method using Test.startTest() / stopTest() block.

◆ 3. Queueable Apex Test

◆ Queueable Class

```
apex

public class QueueableExample implements Queueable {
    public void execute(QueueableContext context) {
        System.debug('Running queueable job');
    }
}
```

◆ Test Class

```
apex

@isTest
private class TestQueueableExample {
    @isTest
```

```
static void testQueueableExecution() {
    Test.startTest();
    System.enqueueJob(new QueueableExample());
    Test.stopTest();

    // Again, mostly just verifying enqueue doesn't throw errors
}
}
```

✓ Tests the queueable job is enqueued and executes successfully.

◆ 4. Scheduled Apex Test

◆ Scheduled Class

```
apex

public class MyScheduledJob implements Schedulable {
    public void execute(SchedulableContext context) {
        System.debug('Scheduled job ran');
    }
}
```

◆ Test Class

```
apex

@isTest
private class TestScheduledJob {
    @isTest
    static void testScheduledExecution() {
        String cron = '0 0 12 * * ?'; // Run at 12 PM every day

        Test.startTest();
        System.schedule('Test Job', cron, new MyScheduledJob());
        Test.stopTest();

        // Test passes if no exception is thrown
    }
}
```

✓ Tests that the scheduled job is successfully scheduled.

◆ 5. REST API Callout Test with Mock

◆ Class Making HTTP Callout

```
apex

public class ExternalService {
    public static String getData() {
        Http http = new Http();
        HttpRequest req = new HttpRequest();
        req.setEndpoint('https://example.com/data');
        req.setMethod('GET');

        HttpResponse res = http.send(req);
        return res.getBody();
    }
}
```

```
}
}
```

◆ Mock Class

```
apex

@isTest
global class MockHttpResponseGenerator implements HttpCalloutMock {
    global HTTPResponse respond(HTTPRequest req) {
        HTTPResponse res = new HTTPResponse();
        res.setHeader('Content-Type', 'application/json');
        res.setBody('{"message": "Success"}');
        res.setStatusCode(200);
        return res;
    }
}
```

◆ Test Class

```
apex

@isTest
private class TestExternalService {
    @isTest
    static void testGetData() {
        Test.setMock(HttpCalloutMock.class, new MockHttpResponseGenerator());

        String response = ExternalService.getData();
        System.assertEquals('{"message": "Success"}', response);
    }
}
```

✓ Tests that a callout returns the mock response using `HttpCalloutMock`.

Summary Table

Concept	Tools Used	Key Points
Trigger + Handler	insert, assertEquals	Validates business logic in trigger
Future Method	@future, Test.startTest()	Validates async method runs
Queueable	System.enqueueJob()	Confirms job enqueues
Scheduled Apex	System.schedule()	Ensures schedule runs without errors
HTTP Callout	HttpCalloutMock, Test.setMock()	Mocks external API and tests response

Would you like a downloadable `.cls` or `.txt` version of all these examples? Or more examples for **batchable chained jobs, platform events, or Apex REST services**?